CS675
Computer Graphics

Interpolation for Animation

Parag Chaudhuri



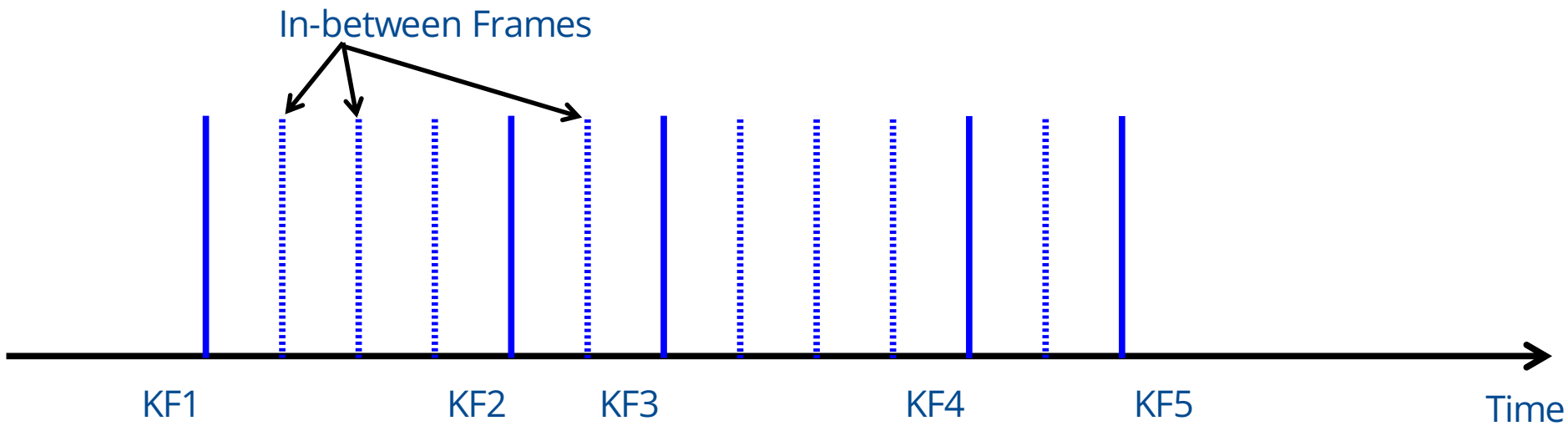
Department of Computer Science and Engineering
IIT Bombay



ViGIL
VISION, GRAPHICS,
IMAGING & LEARNING

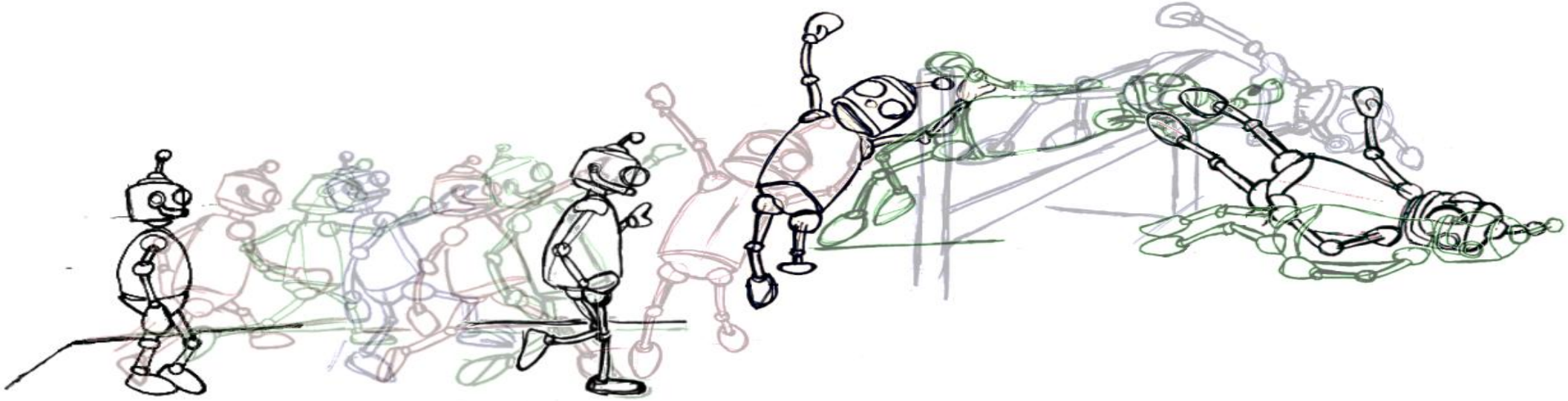
Animation

- Keyframing
  - Selected (key) frames are specified.
  - Interpolation of intermediate frames.
  - Simple and popular approach.
  - May give incorrect (inconsistent) results.



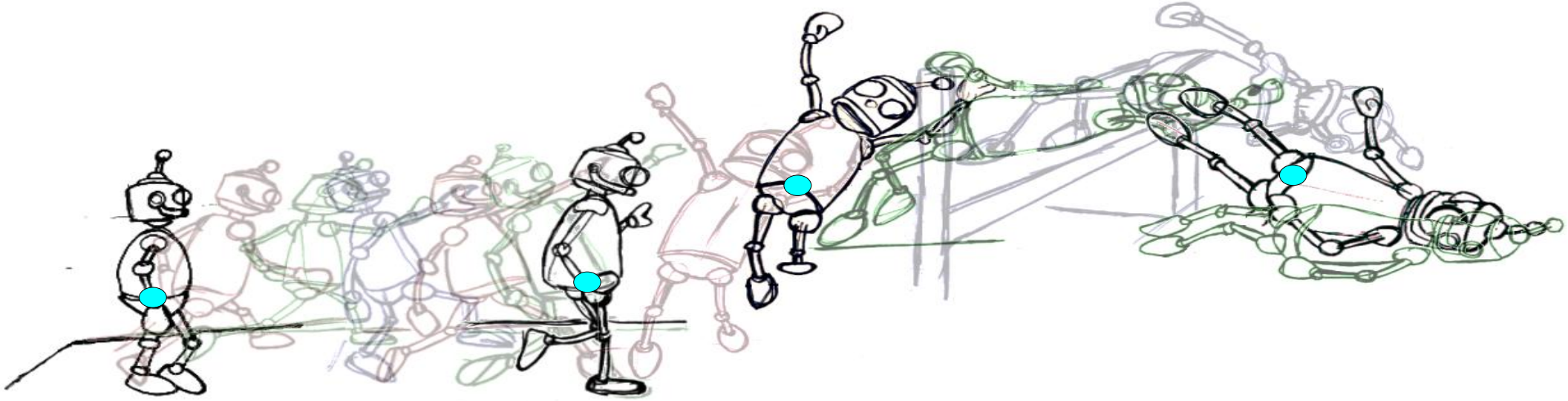
Keyframing

- Also called *tweening* (short for in-betweening)



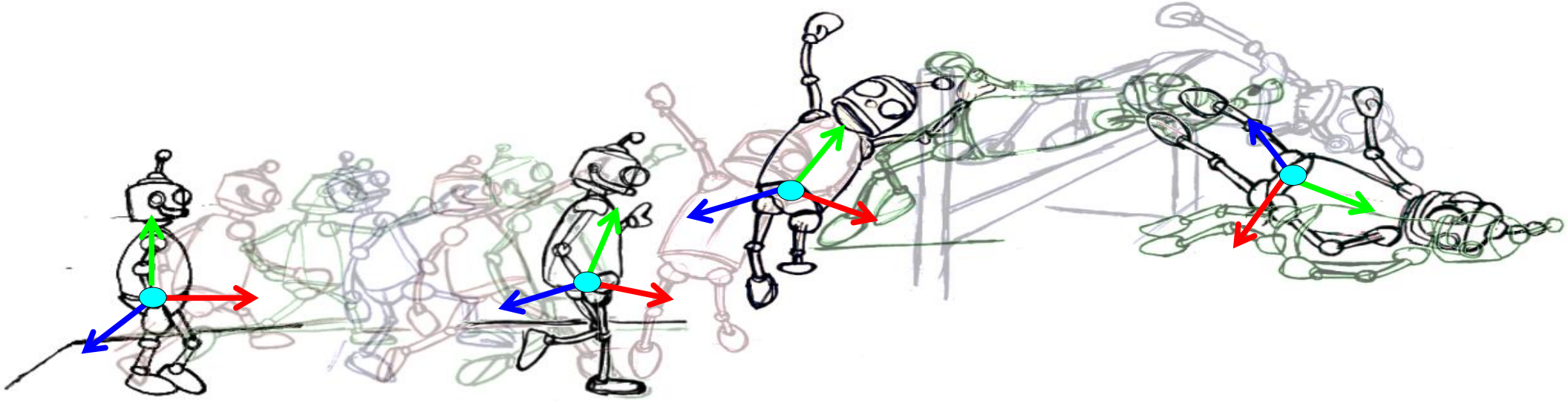
Keyframed Animation

- Interpolate Position



Keyframed Animation

- Interpolate Orientation



Keyframed Animation

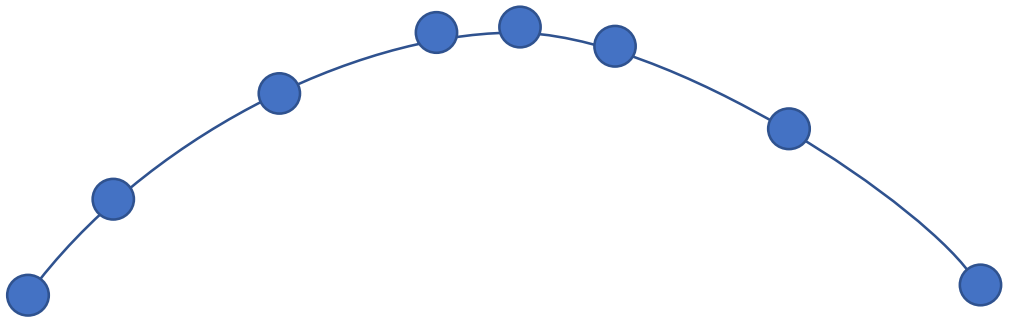
- Interpolate Orientation
- Interpolate Position
- Interpolate Shape
- Interpolate Colour
- Light Intensity
- Camera Zoom
- Any other parameter

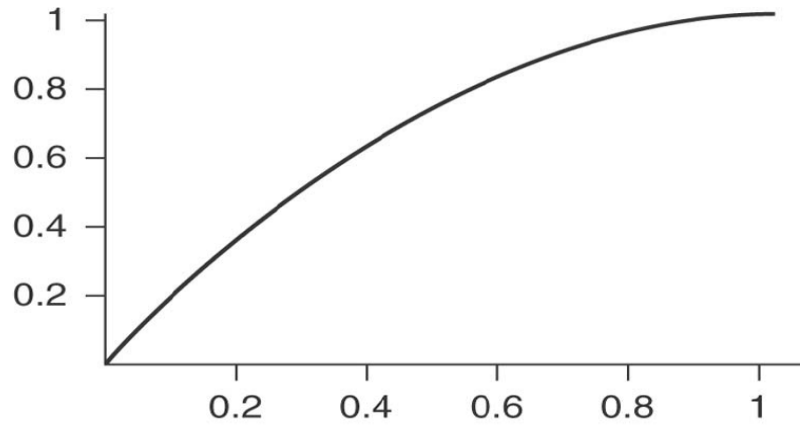


Real-Time Shape Editing using Radial Basis Functions. Mario Botsch, Leif Kobbelt. Computer Graphics Forum 24(3), Proc. Eurographics 2005

Interpolating Position

- Interpolate position by using parametric curves
  - Moving on curves.
    - Specify spatial position to fix the curve – a trajectory
    - Specify a speed profile – a s-t curve.
    - Animate.





Reparameterization of Curves

- To find position on the curve from distance travelled.
- Distance travelled on the curve is called Arc Length

$$\text{LENGTH}(u_1, u_2) = s = \int_{u_1}^{u_2} \left\| \frac{dP(u)}{du} \right\| du = \int_{u_1}^{u_2} \sqrt{\left(\frac{dP(u)}{du}\right)^2}\, du$$

$$\sqrt{\left(\frac{dP(u)}{du}\right)^2} = \sqrt{\left(\frac{dx(u)}{du}\right)^2 + \left(\frac{dy(u)}{du}\right)^2 + \left(\frac{dz(u)}{du}\right)^2}$$

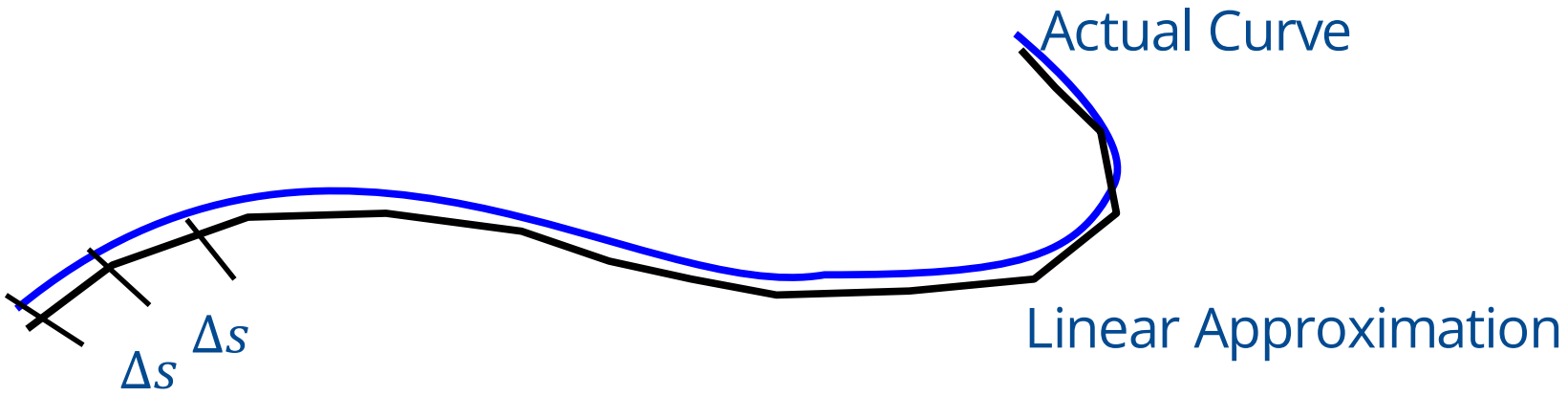
Arc Length Reparameterization

- Can we reparametrize such that the parameter used is arc length?
- For this we must first be able to compute arc length:
  - Analytical Computation
  - Summed linear distances (forward differencing)
  - Gaussian quadrature (numerical integration)



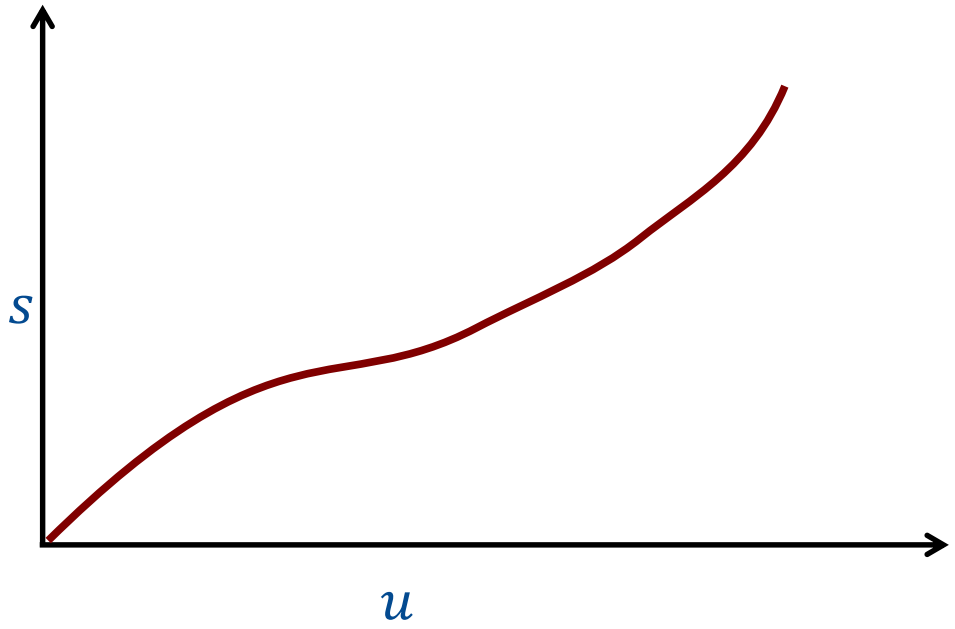
Computing Arc Length

- The arc length integral can be approximated using a forward differencing method.
- Given a parametric curve, P(u) = (x(u), y(u), z(u))
- Create a piece wise linear approximation of the curve from many parameter evaluations and sum these to form the arc length.
- Store these values into a table – so we have a table between s and u.
- This represents a function s = G(u)



Computing Arc Length

We can also compute an inverse function for $u = G^{-1}(s)$



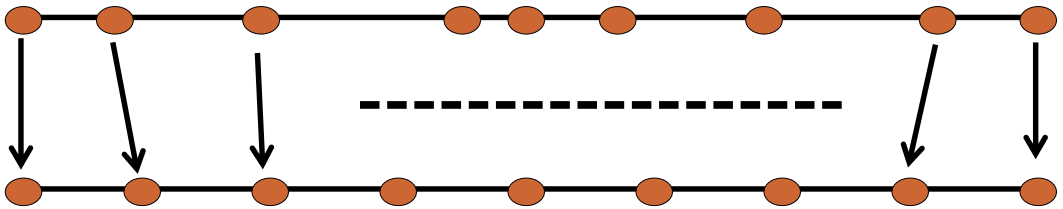
$s = G(u)$
is a monotonically increasing function, i.e., if $u_1 < u_2$ then $s_1 < s_2$

So we can do a bisection or a binary search for u, given a value of s to lookup the inverse.

Arc Length Reparameterization

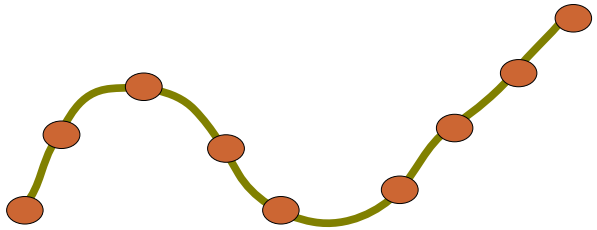
- With G, arc length parametrization can be obtained by inversion as $P(G^{-1}(s))$, where $G^{-1}(s)$ gives the parameter u up to which distance travelled on the curve is s.
- Now we just say that a new parameter t=s can be used, to get u via the inverse and then the point on the curve. Equal increments in t will produce equal arc lengths over the curve.
- Thus t is an arc length parameterization

Parameter values u obtained from $G^{-1}(s)$



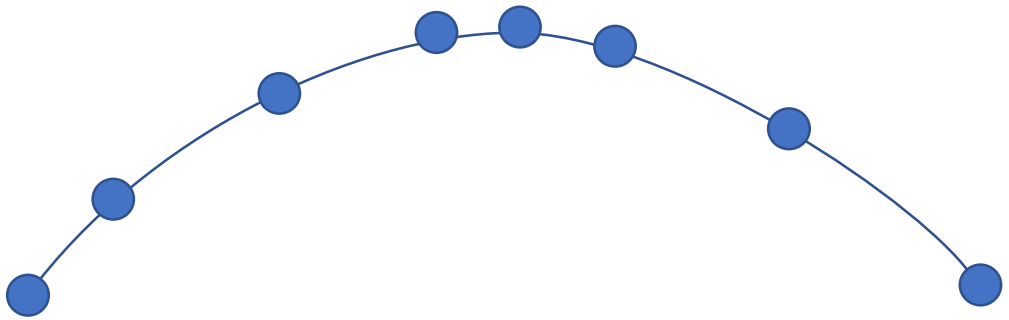
Re-parametrize to t have equal spacing in the parametric interval.

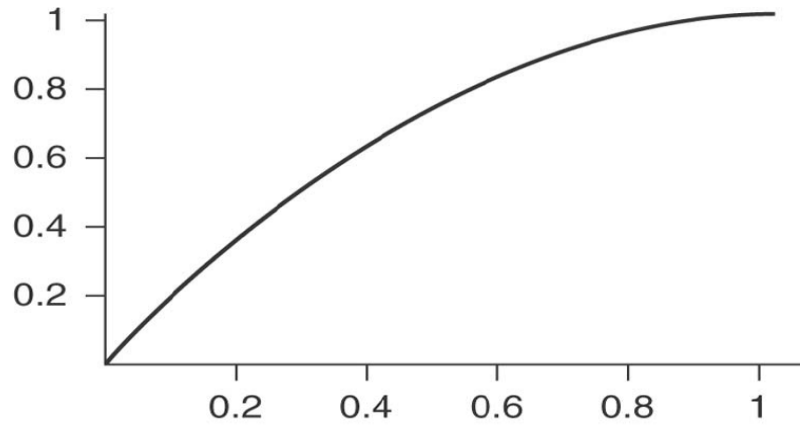
Equal arc lengths s over the curve.



Interpolating Position

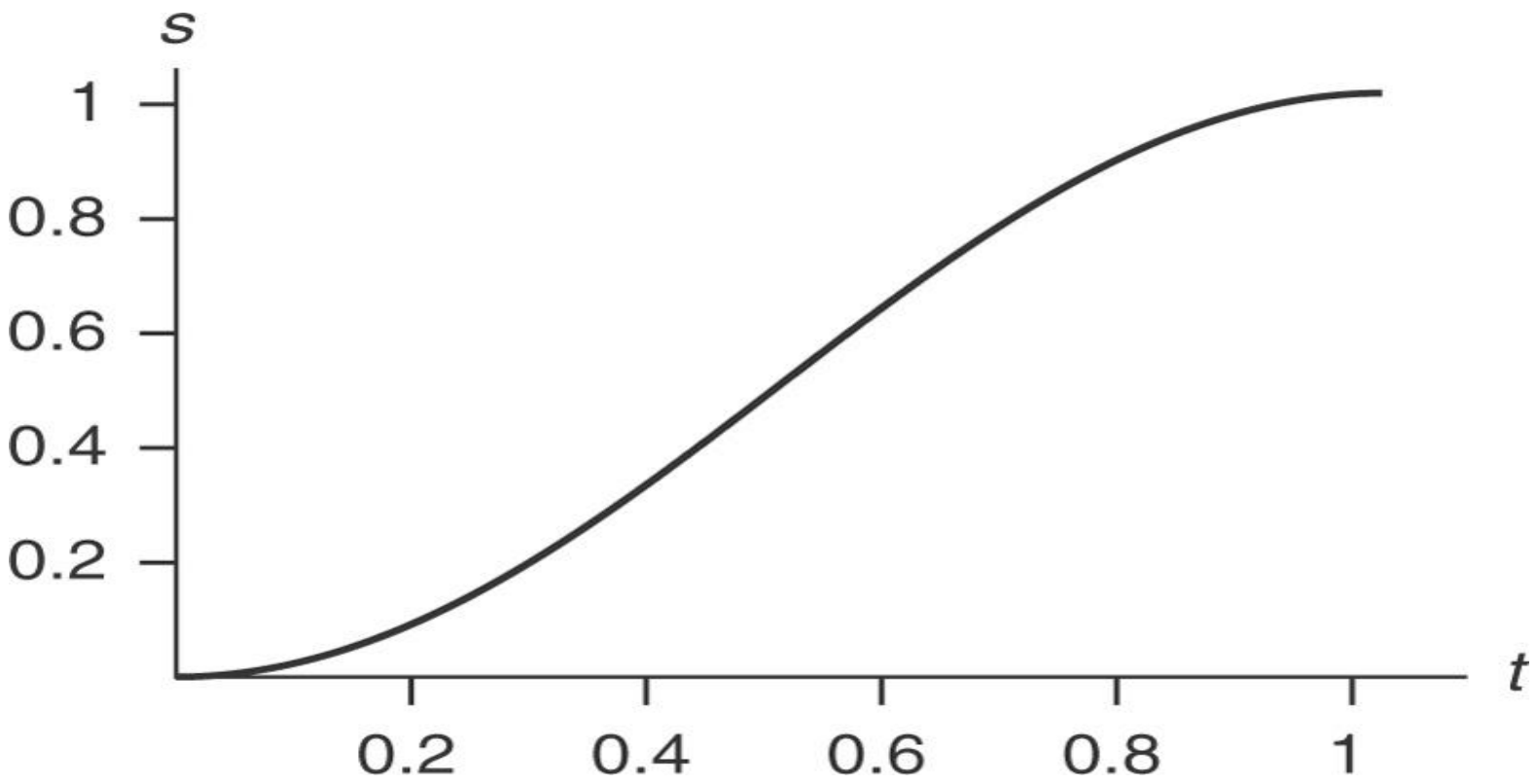
- Interpolate position by using parametric curves
  - Moving on curves.
    - Specify spatial position to fix the curve – a trajectory
    - Specify a speed profile – a s-t curve.
    - Animate.





Controlling speed on curves

- A slow in – slow out curve may look like:

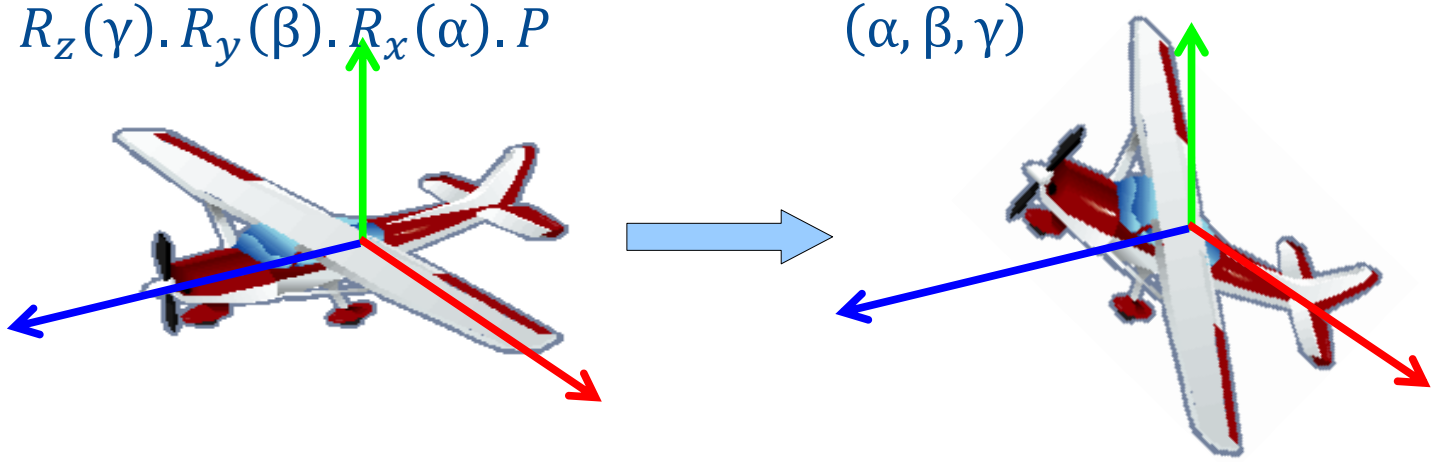


Interpolating Orientation

- Fixed Axis Representation - Ordered triple of rotations about *global axes.*
- Any triple is valid that doesn't immediately repeat an axis, e.g., x-y-z or z-x-y. But not x-x-y.
- Let us assume a z-y-x order for now.

$$P' = R_z(\gamma).R_y(\beta).R_x(\alpha).P$$

(α, β, γ)

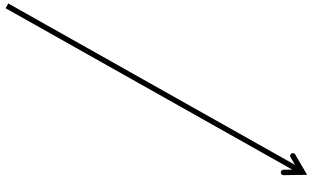


Interpolating rotation matrices

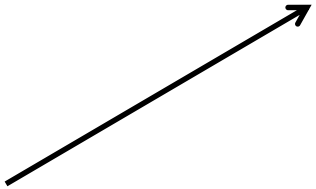
Make a rotation matrix from the angles and interpolate

$$R_z(90) = \begin{bmatrix} 0 & -1 & 0 & 0 \\ 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

$$R_z(-90) = \begin{bmatrix} 0 & 1 & 0 & 0 \\ -1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

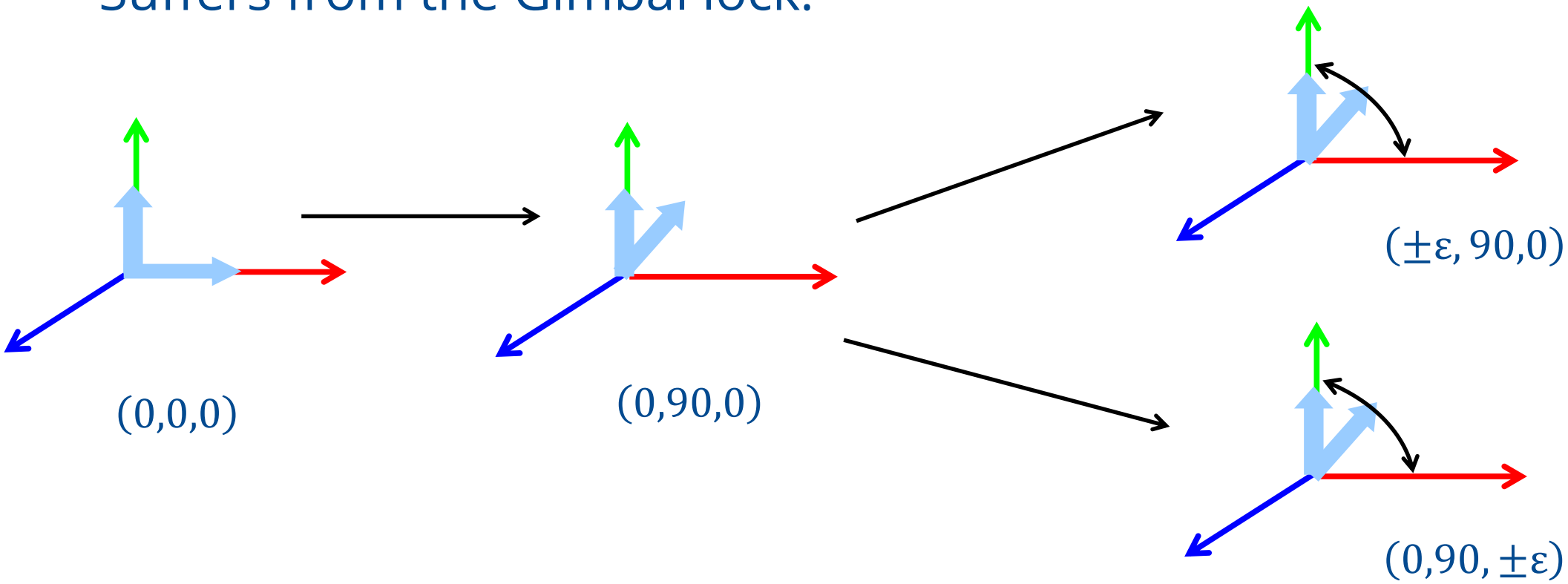


$$R_z(?) = \begin{bmatrix} 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$



Interpolating orientation

- Fixed Axis Representation
- Interpolate the angles and then form the matrix.
- Suffers from the Gimbal lock!



Gimbal Lock

A rotation in 3D space can be represented numerically with matrices in several ways. One of these representations is:

$$R = \begin{bmatrix} 1 & 0 & 0 \\ 0 & \cos\alpha & -\sin\alpha \\ 0 & \sin\alpha & \cos\alpha \end{bmatrix} \begin{bmatrix} \cos\beta & 0 & \sin\beta \\ 0 & 1 & 0 \\ -\sin\beta & 0 & \cos\beta \end{bmatrix} \begin{bmatrix} \cos\gamma & -\sin\gamma & 0 \\ \sin\gamma & \cos\gamma & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

An example worth examining happens when $\beta = \frac{\pi}{2}$. Knowing that $\cos\frac{\pi}{2} = 0$ and $\sin\frac{\pi}{2} = 1$, the above expression becomes equal to:

$$R = \begin{bmatrix} 1 & 0 & 0 \\ 0 & \cos\alpha & -\sin\alpha \\ 0 & \sin\alpha & \cos\alpha \end{bmatrix} \begin{bmatrix} 0 & 0 & 1 \\ 0 & 1 & 0 \\ -1 & 0 & 0 \end{bmatrix} \begin{bmatrix} \cos\gamma & -\sin\gamma & 0 \\ \sin\gamma & \cos\gamma & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

Carrying out matrix multiplication:

$$R = \begin{bmatrix} 0 & 0 & 1 \\ \sin\alpha & \cos\alpha & 0 \\ -\cos\alpha & \sin\alpha & 0 \end{bmatrix} \begin{bmatrix} \cos\gamma & -\sin\gamma & 0 \\ \sin\gamma & \cos\gamma & 0 \\ 0 & 0 & 1 \end{bmatrix} = \begin{bmatrix} 0 & 0 & 1 \\ \sin\alpha\cos\gamma + \cos\alpha\sin\gamma & -\sin\alpha\sin\gamma + \cos\alpha\cos\gamma & 0 \\ -\cos\alpha\cos\gamma + \sin\alpha\sin\gamma & \cos\alpha\sin\gamma + \sin\alpha\cos\gamma & 0 \end{bmatrix}$$

And finally using the trigonometry formulas:

$$R = \begin{bmatrix} 0 & 0 & 1 \\ \sin(\alpha+\gamma) & \cos(\alpha+\gamma) & 0 \\ -\cos(\alpha+\gamma) & \sin(\alpha+\gamma) & 0 \end{bmatrix}$$

https://en.wikipedia.org/wiki/Gimbal_lock

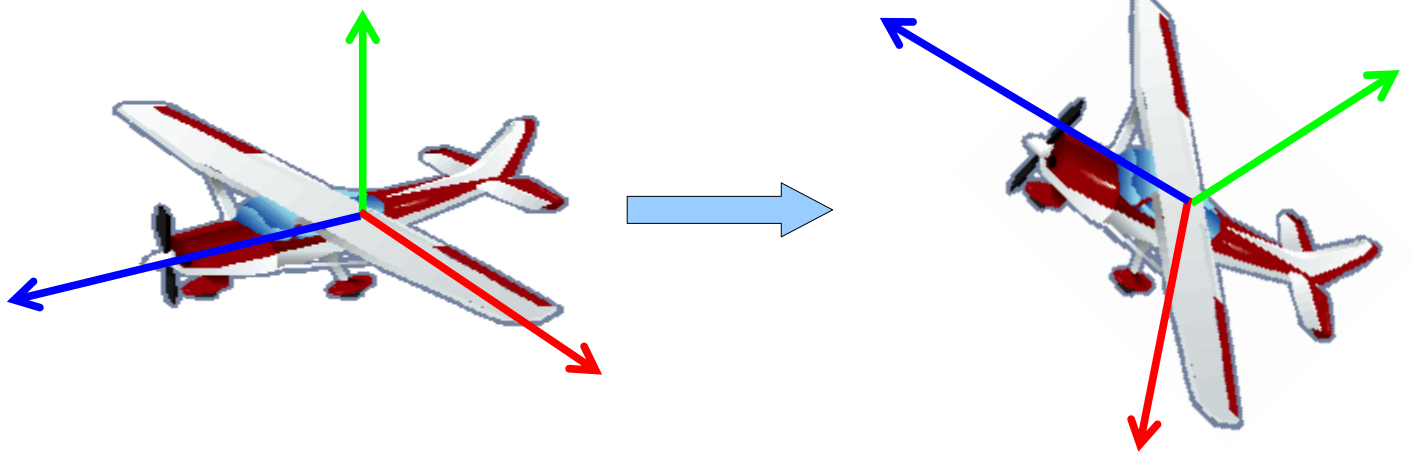
Interpolating orientation

- Moving axis Representation - Ordered triple of rotations about *local axes.*
- Any triple is valid that doesn't immediately repeat an axis, e.g., x-y-z or z-x-y. But not x-x-y.
- Let us assume a x-y-z order for now.

Rotation given by a triad of moving axis angles is the same as given by a triad of fixed axis angles considered in **opposite** order.

So it has the same Gimbal Lock problem.

$$P' = R_x(\alpha).R_y(\beta).R_z(\gamma).P$$



Interpolating orientation

- When to form and apply the matrix if rotation Θ has to be incremented by $\delta\theta$ in each frame?

1. Form a rotation matrix for $\delta\theta$ and apply repeatedly to rotated object in each frame.

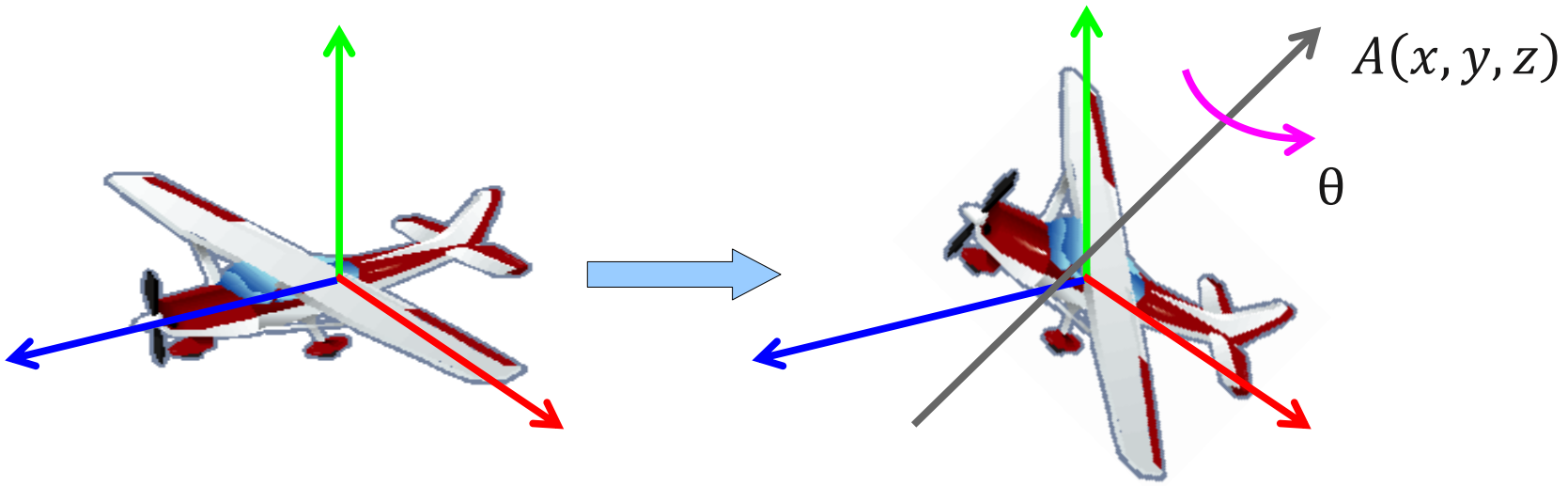
2. Update the rotation matrix $R_{axis}(\Theta)$ by multiplying with $R_{axis}(\delta\theta)$ in each frame. Apply updated matrix to the object.

3. Update the rotation angle, Θ by the increment $\delta\theta$ and form the new matrix $R_{axis}(\Theta + \delta\theta)$ in each frame. Apply this matrix to the object.

Axis Angle Representation

- It is specified as an axis of rotation $A(x, y, z)$ and an angle of rotation, θ around it.

- Euler's Theorem – Any orientation can be derived from another by a single rotation about and axis.



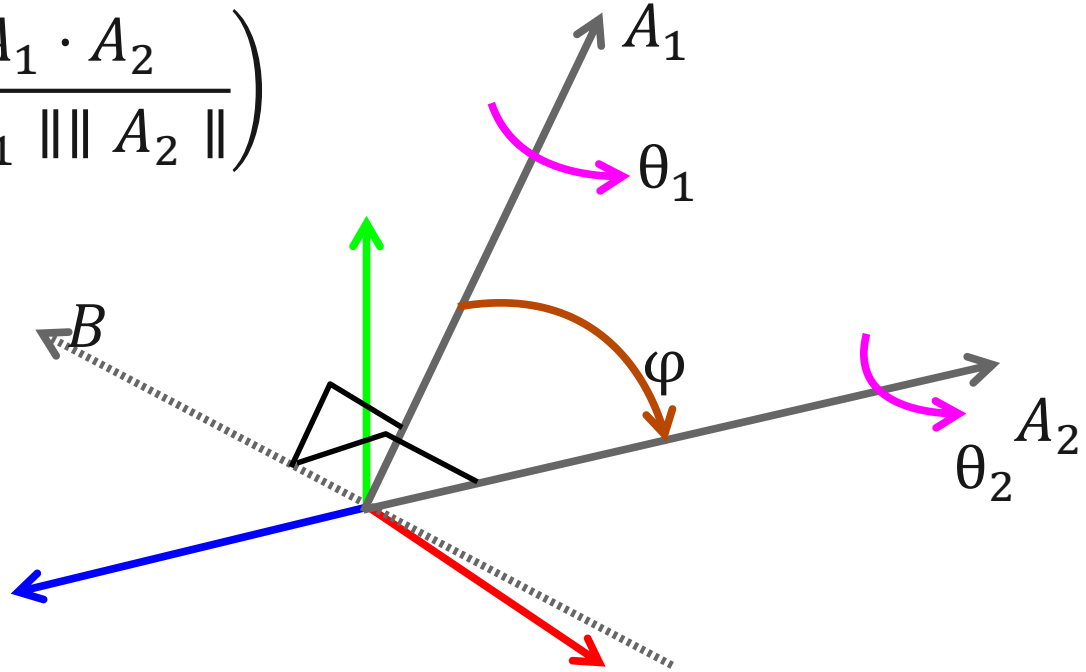
Axis Angle Representation

- Interpolating between two orientations (A_1, θ_1) *and* (A_2, θ_2)

$$B = A_1 \times A_2 \qquad \varphi = \cos^{-1}\left(\frac{A_1 \cdot A_2}{\| A_1 \| \| A_2 \|}\right)$$

$$A_k = R_B(k.\varphi)A_1$$
$$\theta_k = (1-k)\theta_1 + k\theta_2$$

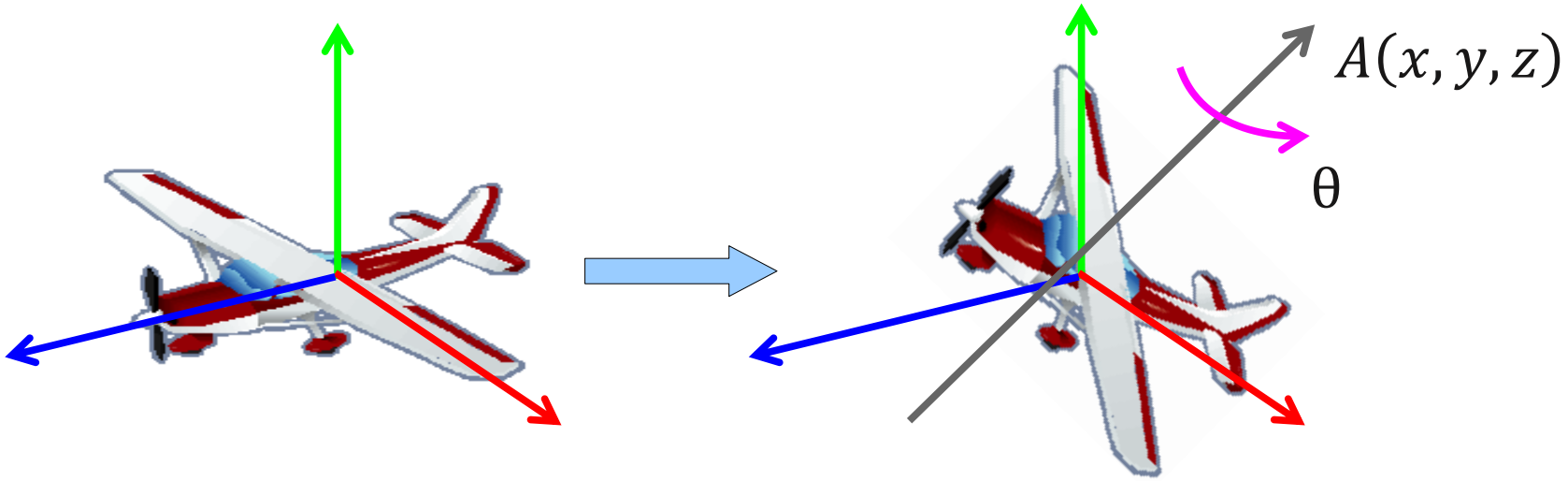
with $0 \leq k \leq 1$



Quaternions

- Have the same information as the axis-angle representation but in a more convenient form.

$$q = [s, x, y, z] = [s, v] = \left[\cos \theta/2, \sin \theta/2 * \hat{a}\right], where\ \hat{a} = \frac{A}{\| A \|}$$



Quaternions

- Quaternions are a non commutative number system that extends complex numbers
- Defined as: $q = s + x\hat{\imath} + y\hat{\jmath} + z\hat{k} = [s, u]$
- where $1, \hat{\imath}, \hat{\jmath}, \hat{k}$ are called the Hamilton basis
- The product of the basis elements is defined as:
- $\hat{\imath}^2 = \hat{\jmath}^2 = \hat{k}^2 = \hat{\imath}\,\hat{\jmath}\,\hat{k} = -1$
- $\hat{\imath}\,\hat{\jmath} = \hat{k}$, $\hat{\jmath}\,\hat{k} = \hat{\imath}$, $\hat{k}\,\hat{\imath} = \hat{\jmath}$
- $\hat{\jmath}\,\hat{\imath} = -\hat{k}$, $\hat{k}\,\hat{\jmath} = -\hat{\imath}$, $\hat{\imath}\,\hat{k} = -\hat{\jmath}$

Quaternions

- Quaternions are a non commutative number system that extends complex numbers
- Defined as: $q = s + x\hat{\imath} + y\hat{\jmath} + z\hat{k} = [s, u]$
- where $1, \hat{\imath}, \hat{\jmath}, \hat{k}$ are called the Hamilton basis
- The product of the basis elements is defined as:
- $\hat{\imath}^2 = \hat{\jmath}^2 = \hat{k}^2 = \hat{\imath}\,\hat{\jmath}\,\hat{k} = -1$
- $\hat{\imath}\,\hat{\jmath} = \hat{k}$, $\hat{\jmath}\,\hat{k} = \hat{\imath}$, $\hat{k}\,\hat{\imath} = \hat{\jmath}$
- $\hat{\jmath}\,\hat{\imath} = -\hat{k}$, $\hat{k}\,\hat{\jmath} = -\hat{\imath}$, $\hat{\imath}\,\hat{k} = -\hat{\jmath}$

Note that the multiplication being defined here is a quaternion multiplication and not the inner or out product of vectors. It is not commutative.

Quaternions

- Quaternion Arithmetic
- Addition: $q_1 + q_2 = [s_1 + s_2, v_1 + v_2]$
- Scalar Multiplication: $kq = [ks, kv] = ks + kx\hat{\imath} + ky\hat{\jmath} + kz\hat{k}$
- Quaternion Multiplication:

$$
\begin{aligned}
q_1 q_2 &= \left(a_1 + b_1\hat{\imath} + c_1\hat{\jmath} + d_1\hat{k}\right)\left(a_2 + b_2\hat{\imath} + c_2\hat{\jmath} + d_2\hat{k}\right) \\
&= a_1\left(a_2 + b_2\hat{\imath} + c_2\hat{\jmath} + d_2\hat{k}\right) + b_1\hat{\imath}\left(a_2 + b_2\hat{\imath} + c_2\hat{\jmath} + d_2\hat{k}\right) + \\
&\qquad c_1\hat{\jmath}\left(a_2 + b_2\hat{\imath} + c_2\hat{\jmath} + d_2\hat{k}\right) + d_1\hat{k}\left(a_2 + b_2\hat{\imath} + c_2\hat{\jmath} + d_2\hat{k}\right) \\
&= (a_1a_2) - (b_1b_2 + c_1c_2 + d_1d_2) + (a_1b_2 + b_1a_2 + c_1d_2 - d_1c_2)\hat{\imath} + \\
&\qquad (a_1c_2 - b_1d_2 + c_1a_2 + d_1b_2)\hat{\jmath} + (a_1d_2 + b_1c_2 - c_1b_2 + d_1a_2)\hat{k}
\end{aligned}
$$

Quaternions

- Quaternion Arithmetic
- Conjugate Quaternion: $q^* = s - x\hat{\imath} - y\hat{\jmath} - z\hat{k}$
- Quaternion Norm: $\|q\| = \sqrt{qq^*} = \sqrt{q^*q} = \sqrt{s^2 + x^2 + y^2 + z^2}$
- If α is real then, $\|\alpha q\| = |\alpha|\|q\|$
- The norm is multiplicative: $\|pq\| = \|p\|\|q\|$
- Unit Quaternion: $\hat{q} = {}^{q}/_{\|q\|}$
- Quaternion Inverse: $q^{-1} = {}^{q^*}/_{\|q\|^2}$

Quaternions

Quaternions and the Geometry of $\mathbb{R}^3$

- $\hat{\imath}, \hat{\jmath}, \hat{k}$ denote both the basis vectors of H and a basis for $\mathbb{R}^3$
- Vectors in $\mathbb{R}^3$ can be written as pure imaginary quaternions

$$v = 0 + x\hat{\imath} + y\hat{\jmath} + z\hat{k} = [0, u]$$

- Inner product of vectors in

$$v_1 \cdot v_2 = x_1x_2 + y_1y_2 + z_1z_2 = \frac{1}{2}\left(v_1^* v_2 + v_2^* v_1\right) = \frac{1}{2}\left(v_1 v_2^* + v_2 v_1^*\right)$$

- Cross product of vectors in

$$v_1 \times v_2 = \frac{1}{2}\left(v_1 v_2 - v_2^* v_1^*\right)$$

- Quaternion multiplication can be written as:

$$q_1 q_2 = [s_1 s_2 - v_1 \cdot v_2, s_2 v_1 + s_1 v_2 + v_1 \times v_2]$$

Unit Quaternions

- A unit quaternion denotes a rotation by an angle θ about an axis A
- $$q = [s, x, y, z] = [s, v] = [cos\,\theta/2\,, sin\,\theta/2 * \hat{a}], \text{where } \hat{a} = \frac{A}{\|A\|}$$
- Multiplication with a unit quaternion q can be used to rotate a vector v

$$R_q(v) = qvq^{-1} = q[0, v]q^{-1}$$

- Composition of rotations is equivalent to quaternion multiplication

$$R_{q_1}\left(R_{q_2}(\vec{v})\right) = R_{q_1}(q_2\vec{v}q_2^{-1}) = q_1q_2[0, v]q_2^{-1}q_1^{-1} = (q_1q_2)[0, v](q_1q_2)^{-1} = R_{q_1q_2}(\vec{v})$$

- Rotating by a scalar multiple of a unit quaternion is the same as rotating by the unit quaternion $R_q(\vec{v}) = R_{kq}(\vec{v})$
  - Only Unit Quaternions represent rotations!

Unit Quaternions

- Antipodal Unit Quaternions
- $$q = [cos(\theta/2), \hat{a}sin(\theta/2)]$$
- If we rotate by θ-2π instead of θ

$$[cos((\theta - 2\pi)/2), \hat{a}sin((\theta - 2\pi)/2)]$$
$$= [cos(\theta/2 - \pi), \hat{a}sin(\theta/2 - \pi)] = [-cos(\theta/2), -\hat{a}sin(\theta/2)] = -q$$

- So both $q \text{ and } -q$ represent the same rotation and are called antipodal points.

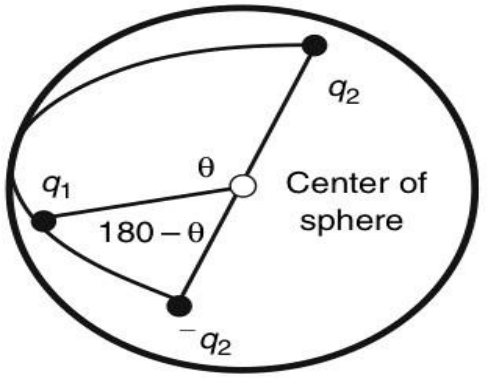
$$R_q(v) = R_{-q}(v)$$

Interpolating Quaternions

- Linear Interpolation $q = (1-k)q_1 + kq_2$
- How to take equi-distant steps along orientation path?
- How to pass through orientations smoothly?
- With dual unit quaternion representations, which o

Dual representation: For Interpolation between q_1 and q_2, compute cosine between q_1 and q_2 and between q_1 and $-q_2$; choose smallest angle.

If $0 < \theta < \pi$ then the positive rotation is the shorter one else the negative rotation is the shorter one, i.e., the quaternion with the positive value of the s coordinate will give the shorter path



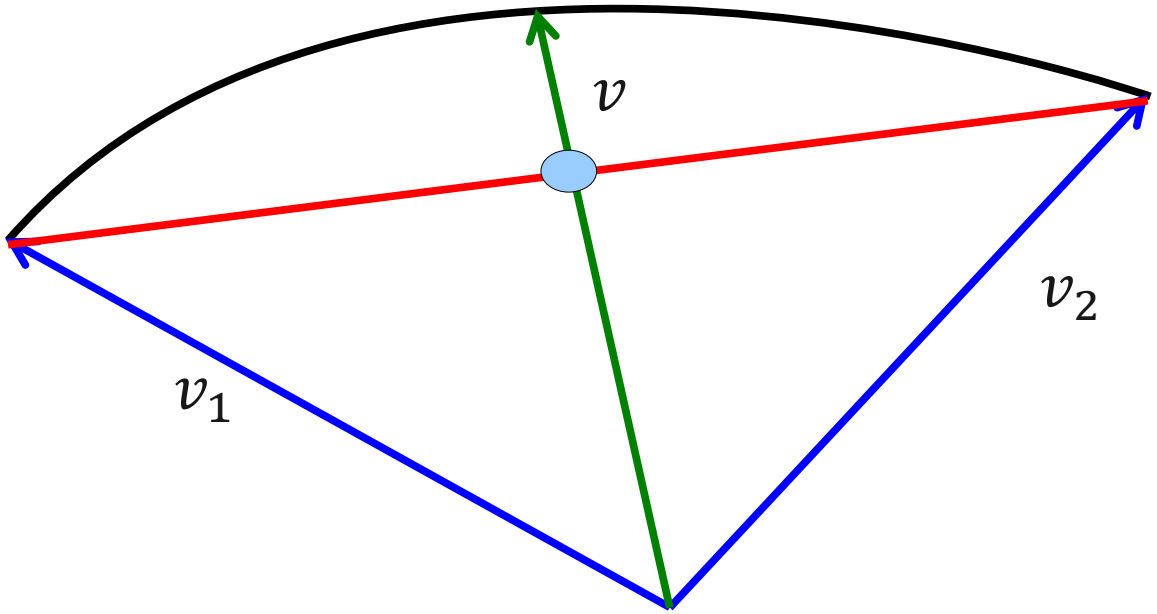
Computer Animation: Algorithms and Techniques, Rick Parent, Morgan Kaufmann, 2001

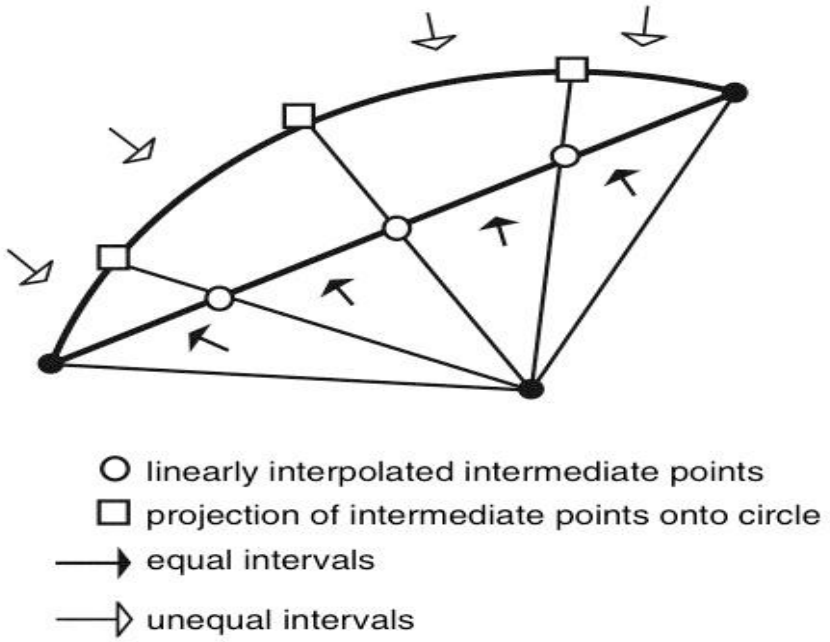
Spherical Linear Interpolation

- Linear Interpolation between unit vectors v_1 and v_2 is given by

$$v = (1 - k)v_1 + kv_2$$

- This is not equally spaced in rotation space.





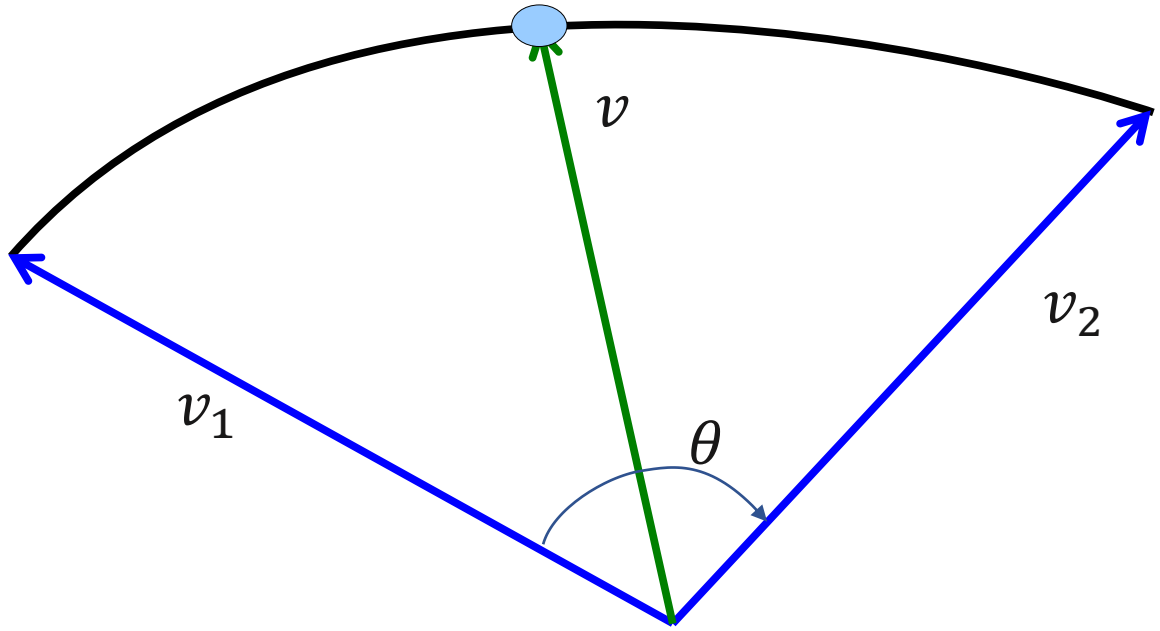
Computer Animation: Algorithms and Techniques, Rick Parent, Morgan Kaufmann, 2001

Spherical Linear Interpolation

SLERP

$$Slerp(v_1, v_2, \alpha) = \frac{\sin(1-\alpha)\theta}{\sin\theta} v_1 + \frac{\sin(\alpha)\theta}{\sin\theta} v_2$$

$$\cos\theta = v_1 \cdot v_2$$



SLERP also applies similarly to unit quaternions.

Spherical Linear Interpolation

- We write, $q^{\alpha} = [cos(\alpha\,\theta/2), \hat{a}\,sin(\alpha\,\theta/2)]$
- We want to interpolate between two rotations $q_1 \text{and} q_2$
- Rotation that takes us from 1 to 2 is given by $q_2 q_1^{-1}$
- Now we start at 1, and go to 2 in α steps as $(q_2 q_1^{-1})^{\alpha} q_1$

$$Slerp(q_1, q_2, \alpha) = (q_2 q_1^{-1})^{\alpha} q_1$$

- We can also generalize from 3D vectors

$$Slerp(q_1, q_2, \alpha) = \frac{sin(1-\alpha)\theta}{sin\theta} q_1 + \frac{sin(\alpha)\theta}{sin\theta} q_2$$

$$cos\theta = q_1 \cdot q_2 = s_1 s_2 + x_1 x_2 + y_1 y_2 + z_1 z_2$$